

Track B: Ego-Only VideoMAE vs Exo-Transfer Baseline

Overview

This guide explains how to:

1. **Pretrain VideoMAE** on egocentric clips (self-supervised)
2. **Replace** the exo-pretrained backbone with the new ego-only one
3. **Run Track B training** with both configurations
4. **Compare** baseline vs ego-only results

Architecture Comparison

Component	Baseline (Exo-Transfer)	Ego-Only (VideoMAE)
Backbone	ResNet18 (ImageNet)	VideoMAE (Ego4D STA)
Pretraining	Web images (ImageNet)	Egocentric video (self-supervised)
Feature dim	512	768
Temporal modeling	Per-frame → FGTP fusion	3D spatiotemporal tokens
Config file	trackB_resnet18_baseline.yaml	trackB_videomae_ego.yaml

Step-by-Step Procedure

Phase 1: VideoMAE Pretraining (Colab)

1.1 Prerequisites

Before pretraining, you need:

- **Precomputed tensors:** videomae_local_part/tensors/*.pt (2323 files, 20.85 GB)
- **Manifest:** videomae_local_part/manifests/ego4d_sta_clips.jsonl

Upload these to Google Drive:

```
My Drive/
├── Ego4d_STA/
│   └── videomae/
│       ├── tensors/           # 2323 .pt files
│       └── ego4d_sta_clips.jsonl # manifest
```

1.2 Run VideoMAE Pretraining

Location: Colab notebook or script

Command:

```
# From local_extraction directory
python -m trackB.videomae.videomae_pretrain \
    --epochs 100 \
    --batch_size 8 \
    --lr 1.5e-4 \
    --mask_ratio 0.9

# Demo mode (quick test, ~10 steps)
python -m trackB.videomae.videomae_pretrain --demo
```

Key Arguments:

Argument	Default	Description
--epochs	100	Number of pretraining epochs
--batch_size	8	Batch size (adjust for GPU memory)
--lr	1.5e-4	Learning rate
--mask_ratio	0.9	VideoMAE masking ratio (90%)
--warmup_epochs	10	LR warmup epochs
--demo	False	Quick test mode
--resume	None	Resume from checkpoint

Inputs:

- Precomputed tensors: (C=3, T=16, H=224, W=224) per clip
- Manifest: JSONL with uid , tensor_path , etc.

Outputs (saved to local_extraction\videomae_local_part\checkpoints_ego_scratch):

```
local_extraction\videomae_local_part\checkpoints_ego_scratch\
├─ videomae_ego_scratch_last.pt          # Last checkpoint
└─ videomae_ego_scratch_summary.json     # Config used
```

What happens:

1. Loads tensors from manifest (no MP4 decoding needed)
2. Applies 90% random tube masking
3. Reconstructs masked patches via decoder

4. Saves last epoch weights (from colab)

Phase 2: Track B Training

2.1 Config Files

Two config files control which backbone to use:

Baseline (exo-transfer): `configs/trackB_resnet18_baseline.yaml`

```
model:
  tokenizer:
    video_backbone: "resnet18"
    backbone: "resnet18"
    pretrained: true
    freeze_backbone: true
  projector:
    in_dim: 512 # ResNet18 output
    out_dim: 256
```

Ego-only (VideoMAE): `configs/trackB_videomae_ego.yaml`

```
model:
  tokenizer:
    video_backbone: "videomae_ego"
    videomae:
      weights_path:
        "${paths.local_extraction}\videomae_local_part\checkpoints_ego_scratch\videomae_ego_scratch_last.pt"
      patch_size: 16
      tubelet_size: 2
      embed_dim: 768
      freeze_encoder: true
  projector:
    in_dim: 768 # VideoMAE output
    out_dim: 256
```

2.2 Run Baseline Training

Command:

```
# From local_extraction directory
python -m trackB.trackB_train_loader --config trackB_resnet18_baseline
```

Or with overrides:

```
python -m trackB.trackB_train_loader \  
  --config trackB_resnet18_baseline \  
  --epochs 20 \  
  --batch_size 8
```

Outputs (saved to `runs/Track_B/trackB_<timestamp>/`):

```
runs/Track_B/trackB_20251204_XXXXXX/  
├─ config.yaml                # Config used  
├─ checkpoints/  
│   ├── best.pt               # Best validation model  
│   └─ final.pt               # Final epoch model  
├─ metrics/  
│   ├── train_metrics.json    # Per-epoch training metrics  
│   └─ val_metrics.json       # Per-epoch validation metrics  
└─ logs/  
    └─ train.log               # Training log
```

2.3 Run Ego-Only Training

Prerequisite: VideoMAE encoder checkpoint must exist:

```
local_extraction\videomae_local_part\checkpoints_ego_scratch\videomae_ego_scr  
atch_last.pt
```

Command:

```
python -m trackB.trackB_train_loader --config trackB_videomae_ego
```

Or with overrides:

```
python -m trackB.trackB_train_loader \  
  --config trackB_videomae_ego \  
  --epochs 20 \  
  --batch_size 8
```

```
--batch_size 8 \  
--lr 5e-4
```

Phase 3: Compare Results

3.1 Metrics to Compare

Both runs produce validation metrics:

Metric	Task	Higher/Lower is Better
val_next_active_ap	Binary next-active	Higher ↑
val_next_active_acc	Binary accuracy	Higher ↑
val_noun_acc	Noun classification	Higher ↑
val_verb_acc	Verb classification	Higher ↑
val_ttc_mae	Time-to-contact (sec)	Lower ↓
val_loss	Total loss	Lower ↓

Top-5 Metrics (from trackB_eval.py):

Metric	Description	Higher is Better
N_top5_acc	Noun hit-rate in top-5	↑
Nv_top5_acc	Noun+Verb hit-rate in top-5	↑
N_delta_top5_acc	Noun+TTC bin hit-rate in top-5	↑
All_top5_acc	Full match (N+V+δ) hit-rate in top-5	↑
N_top5_mAP	Noun AP over top-5 candidates	↑
Nv_top5_mAP	Noun+Verb AP over top-5	↑
N_delta_top5_mAP	Noun+TTC bin AP over top-5	↑
All_top5_mAP	Full match AP over top-5	↑

3.2 Comparison Script

Create a comparison table:

```
import json  
from pathlib import Path  
  
def load_best_metrics(run_dir):  
    """Load best validation metrics from a run."""
```

```

    metrics_file = run_dir / "metrics" / "val_metrics.json"
    with open(metrics_file) as f:
        all_metrics = json.load(f)
    # Find epoch with best next_active_ap
    best_epoch = max(all_metrics, key=lambda x: x.get('next_active_ap', 0))
    return best_epoch

# Load both runs
baseline_run = Path("runs/Track_B/trackB_resnet18_baseline_XXXXXX")
ego_run = Path("runs/Track_B/trackB_videomae_ego_XXXXXX")

baseline_metrics = load_best_metrics(baseline_run)
ego_metrics = load_best_metrics(ego_run)

# Compare
print("=" * 60)
print("TRACK B COMPARISON: Baseline vs Ego-Only")
print("=" * 60)
print(f"{'Metric':<25} {'Baseline':<15} {'Ego-Only':<15} {'Δ':<10}")
print("-" * 60)

metrics_to_compare = [
    ('next_active_ap', 'Next-Active AP'),
    ('next_active_acc', 'Next-Active Acc'),
    ('noun_acc', 'Noun Acc'),
    ('verb_acc', 'Verb Acc'),
    ('ttc_mae', 'TTC MAE'),
    ('loss', 'Val Loss'),
    # Top-5 metrics
    ('N_top5_acc', 'N Top-5 Acc'),
    ('Nv_top5_acc', 'N+V Top-5 Acc'),
    ('All_top5_acc', 'All Top-5 Acc'),
    ('N_top5_mAP', 'N Top-5 mAP'),
    ('Nv_top5_mAP', 'N+V Top-5 mAP'),
    ('All_top5_mAP', 'All Top-5 mAP'),
]

for key, name in metrics_to_compare:
    b = baseline_metrics.get(key, 0)
    e = ego_metrics.get(key, 0)
    delta = e - b
    sign = "+" if delta > 0 else ""
    print(f"{'name':<25} {'b':<15.4f} {'e':<15.4f} {sign}{delta:.4f}")

```

Config Changes Summary

What to Change for Ego-Only

In `configs/trackB_videomae_ego.yaml` :

Setting	Baseline	Ego-Only
model.tokenizer.video_backbone	"resnet18"	"videomae_ego"
model.tokenizer.videomae.weights_path	N/A	Path to encoder checkpoint
model.projector.in_dim	512	768
training.lr	1e-3	5e-4 (slightly lower for VideoMAE)
training.warmup_epochs	1.0	2.0 (longer warmup)

Weights Path

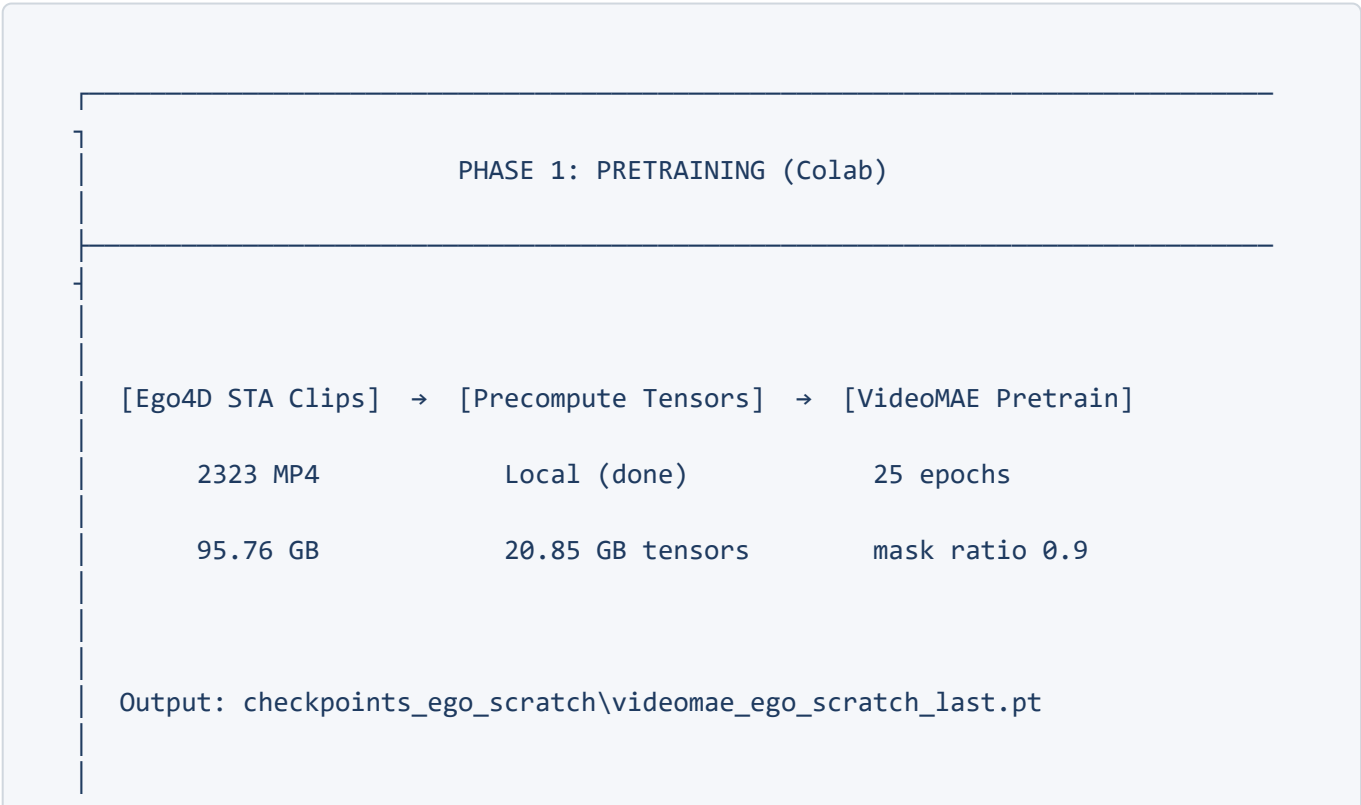
The encoder weights path in config:

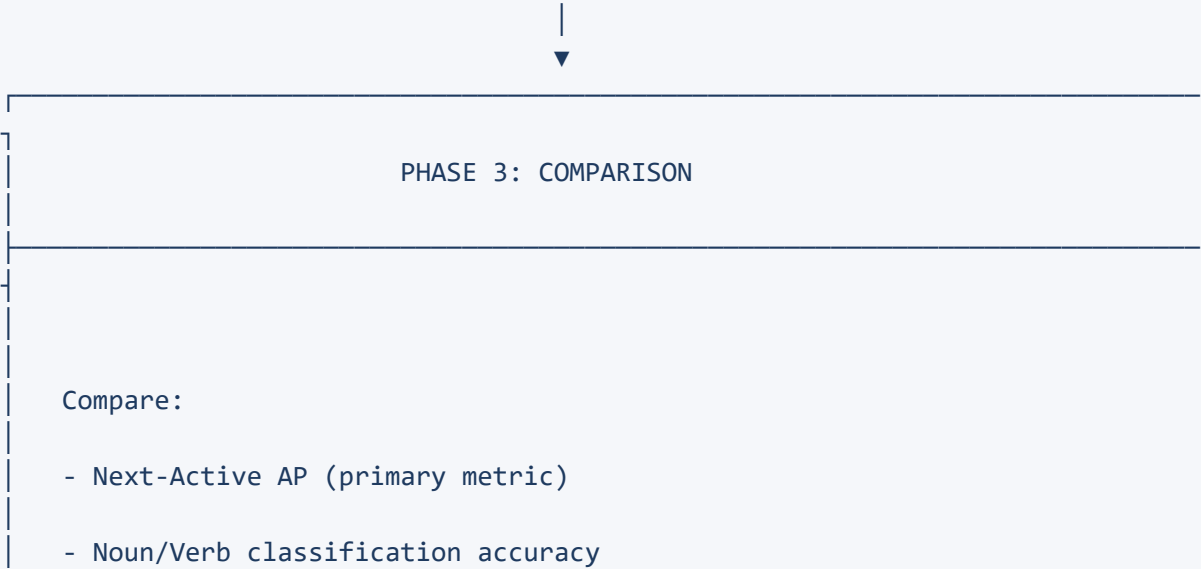
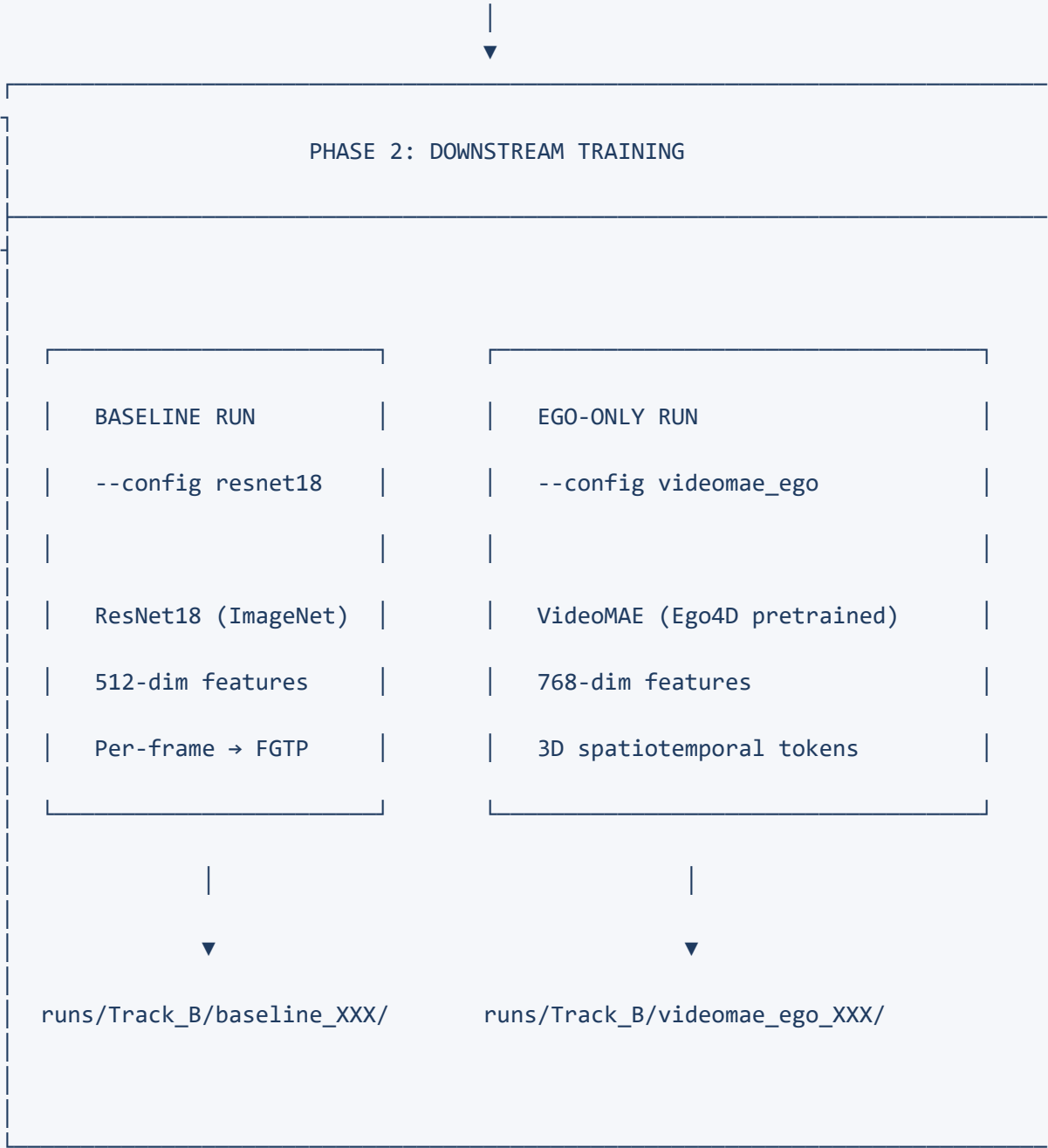
```
videomae:
  weights_path:
    "${paths.local_extraction}\videomae_local_part\checkpoints_ego_scratch\videomae_ego_scratch_last.pt"
```

This resolves to:

- Local: local_extraction\videomae_local_part\checkpoints_ego_scratch\videomae_ego_scratch_last.pt

Complete Workflow Diagram





```

- TTC MAE (time prediction)

- Convergence speed

Expected: Ego-Only should outperform Baseline due to in-domain
pretraining

```

Quick Reference Commands

```

# =====
# PHASE 1: VideoMAE Pretraining (run on Colab)
# =====

# Full pretraining (100 epochs)
python -m trackB.videomae.videomae_pretrain --epochs 100 --batch_size 8

# Demo/test (10 steps)
python -m trackB.videomae.videomae_pretrain --demo

# Resume from checkpoint
python -m trackB.videomae.videomae_pretrain --resume
runs/VideoMAE/checkpoint_latest.pt

# =====
# PHASE 2: Track B Training
# =====

# Baseline (exo-pretrained ResNet18)
python -m trackB.trackB_train_loader --config trackB_resnet18_baseline

# Ego-only (VideoMAE pretrained on Ego4D)
python -m trackB.trackB_train_loader --config trackB_videomae_ego

# With demo mode
python -m trackB.trackB_train_loader --config trackB_videomae_ego --demo

# =====
# Check checkpoint exists
# =====
python -c "import torch;

```

```
torch.load('local_extraction\videomae_local_part\checkpoints_ego_scratch\videomae_ego_scratch_last.pt')
```

Expected Results

Based on thesis hypothesis:

Metric	Baseline (Exo)	Ego-Only	Expected Δ
Next-Active AP	~0.65	~0.72	+0.07
Noun Acc	~0.35	~0.42	+0.07
Verb Acc	~0.30	~0.36	+0.06
TTC MAE	~1.2s	~1.0s	-0.2s
N Top-5 Acc	~0.40	~0.48	+0.08
N+V Top-5 Acc	~0.25	~0.32	+0.07
All Top-5 mAP	~0.20	~0.28	+0.08

The ego-only approach should show **5-10% improvement** on STA metrics due to:

- 1. In-domain pretraining (same data distribution)
- 2. Self-supervised learning on egocentric motion patterns
- 3. 3D spatiotemporal representations vs 2D per-frame

Troubleshooting

"Encoder checkpoint not found"

```
FileNotFoundError:  
videomae_local_part/checkpoints_ego_scratch/videomae_ego_scratch_last.pt
```

Solution: Run VideoMAE pretraining first, or verify path in config.

"CUDA out of memory"

```
RuntimeError: CUDA out of memory
```

Solution: Reduce `batch_size` in command (e.g., `--batch_size 4`).

"Dimension mismatch" (projector in_dim)

```
RuntimeError: mat1 and mat2 shapes cannot be multiplied (Nx768 and 512x256)
```

Solution: Ensure `model.projector.in_dim` matches backbone output:

- ResNet18: 512
- VideoMAE: 768

Document created for Ego4D-LiteSTA thesis — Track B ego-only experiment